

Software Design and Development Coursework (Midterm)

Part 3: Secure programming

The image displays two side-by-side screenshots of a web application interface. The left screenshot shows a login page with a dark blue background. At the top, there is a circular icon with a person silhouette. Below it, the text 'Welcome back!' is displayed. There are two tabs: 'MEMBER' (selected) and 'MANAGER'. The 'EMAIL ADDRESS' field contains 'name@domain.com'. The 'PASSWORD' field is masked with dots. A 'FORGOT PASSWORD?' link is visible. A red 'SIGN-IN' button is at the bottom. The right screenshot shows a 'Signup' page with a dark blue background. At the top, there is a person icon and the text 'Signup'. Below it, there are two sections: 'Member Details' and 'Address'. The 'Member Details' section has a green checkmark icon and a red asterisk indicating required fields. It includes fields for 'FIRST NAME' (Devon), 'LAST NAME' (Xavior), 'GENDER' (dropdown), 'PHONE NUMBER' (+6587892891), 'EMAIL ADDRESS' (devonmayaxavior@gmail.com), 'DATE OF BIRTH' (dd/mm/yyyy), 'ADD SECONDARY PHONE +', 'ADD SECONDARY EMAIL +', and 'PICK A CHECK-IN CODE (4 DIGITS)' (9014). The 'Address' section has a green checkmark icon and includes fields for 'STREET ADDRESS' (Blk 438 Choa Chu Kang Ave 4 #06-485 Singe), 'CITY' (Singapore), 'ZIP / POSTAL CODE' (680438), 'STATE' (Select), and 'COUNTRY' (Singapore).

The web application I've chosen contains a login page that allows 2 types of user to log in (i.e., member or manager). Users use their email addresses and passwords to log into their accounts. The website also has a signup page allowing new users to register for a new account using their email addresses, entering mandatory personal details like first name, last name and optional information like phone numbers, gender and date of birth. One thing to note is that there is a check-in code (4 digits) that verifies the email address of the registrant.

One typical problem with login pages like that of the above is that passwords are stored in plain text in the database. If there's a hack to the database system of the company, the passwords will be readily available for the hackers to copy and manipulate for malicious purposes. A way to avoid that and enhance security is to introduce **Password Hashing**. Passwords are not stored in plain text but encrypted using a hashing algorithm that prevents hackers from identifying the actual passwords to the accounts even if they've managed to breach into the database.

✓ **Account Password**

Please pick a password to log-in to your account later.

PASSWORD (MINIMUM 8 CHARACTERS)

CONFIRM PASSWORD

Another security weakness of the registration page is that there's no strong password enforcement applied. Users are allowed to enter any kind of password (min. 8 characters) they desire for their login. Passwords like that are easily deciphered by hackers and that will threaten the security of the user accounts. To deter this from happening, a **strong password policy** should be mandated during the registration process, requiring caps, special characters plus a mixture of numbers and letters. Passwords should ideally be refreshed by users after a reasonably defined period of time. To further strengthen this, a multi-factor authentication can be introduced as well.

The third way to improve security of the web application is to address the GET method used. Using the GET method means that parameters are usually included in the URL, this can cause security vulnerabilities concerning sensitive information like login addresses, passwords, server logs etc. Additionally, GET requests cached in browsers, actions kept in the cache might be repeated without user knowing. To address this, GET method should be used for actions that don't change server state. For actions concerning sensitive data exchange, **POST method** can be used instead as data requests are sent in the body and not in the URL. As an extra measure, **HTTPS** should be used to encrypt data sending.

Speaking of HTTPS, when an application's connection changes from HTTP to HTTPS (e.g., when authentication is taking place), a new session identifier should be generated. However, as aforementioned, HTTPS should be throughout instead of HTTP for good practice. But in the event that a switch has to occur, **anti-CSRF (Cross-Site Request Forgery)** can also be put in place to prevent attacks of unauthorized actions done by someone other than the authenticated user. Anti-CSRF tokens in forms should be added to determine that requests are from the legit pages.

Last but not least, enforcing proper **Input Validation** will place protection against security problems such as SQL injection and Cross-site scripting (XSS). SQL Injection happens when malicious SQL are added to inputs included in database queries, leading to serious database breaches. In a similar fashion, XSS inputs malicious scripts as well in web pages. As a results, websites may lose sensitive information or vandalism to the pages.

Input validation can prevent the said 2 problems by enforcing secure validation methods in both Client-side and Server-side. Some of the input validation methods are –

Parameterized Queries – This prevents SQL injection by separating user input from SQL query, this method sees user input as data and not codes to be executed.

Input Length Limitations – this limits the amount of data the application can process, hence reducing risks of some attacks.

With the 5 recommended security measures above, the web application can prevent a multitude of potential security vulnerabilities and attacks.